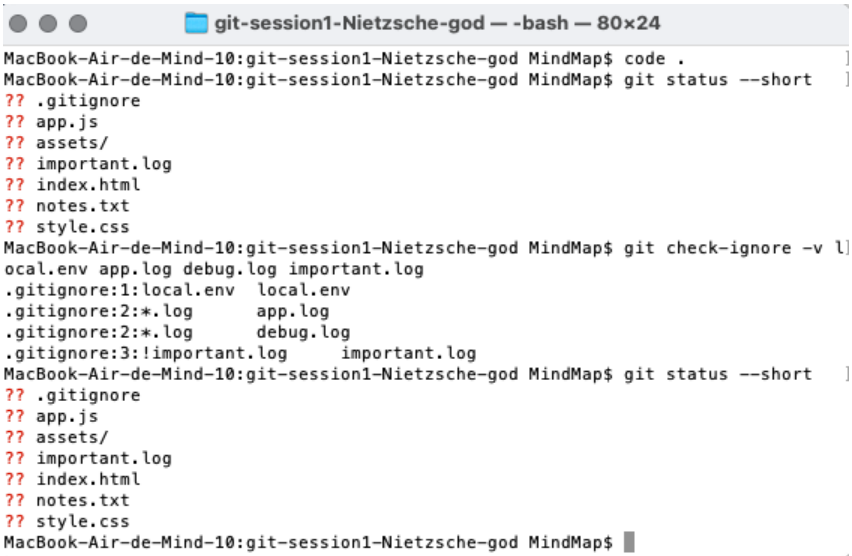
# Assignment 1 - Git & GitHub Fundamentals

Report

| | |
|---|---|
| Full Name: | Mohamed Amine Bouamri |
| Student ID: | 158425 |
| Course Code & Section: | CSC 3324 02 |
| GitHub Username: | Nietzsche-god |
| Repository: | https://github.com/Nietzsche-god/git-session1-Nietzsche-god |

## 1. Repository Setup

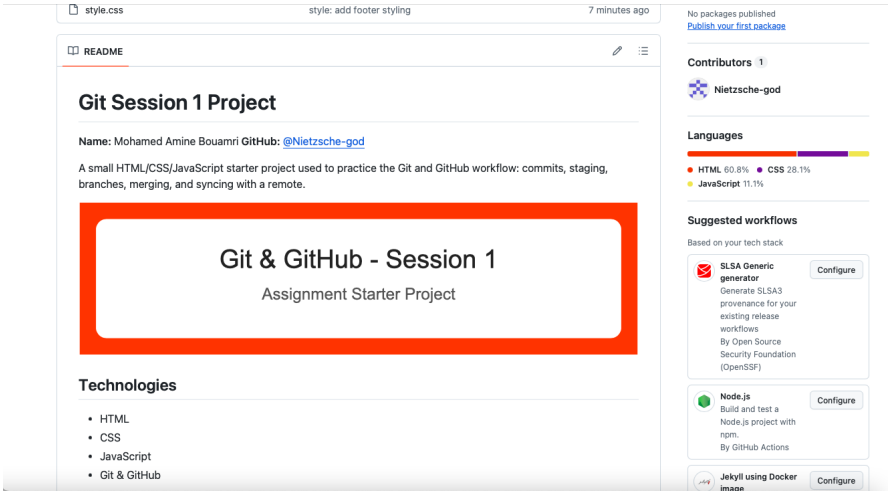
I copied the STARTER files into a folder named `git-session1-Nietzsche-god` and ran `git init` (default branch `main`). My `.gitignore` contains `local.env` and `*.log`, plus the exception `!important.log`. So the local environment file and the normal logs (`app.log`, `debug.log`) are ignored, while `important.log` stays visible to Git. `git check-ignore -v` confirms which rule matches each file, and `git status --short` no longer lists the ignored files.




Ignore check: three files ignored, important.log matched by the negation rule and still untracked.

## 2. README & Commit History

The README contains my name, GitHub username, a short description, a technologies list, one code block (the clone command), one link (my GitHub profile), and the provided banner image. I made small logical commits, each with a prefix describing its purpose (chore, feat, docs, style): (1) .gitignore, (2) starter page files, (3) notes and important.log, (4) README and assets, (5) footer styling, then the About feature, its merge, the README edit made on GitHub, and a final app.js commit. In total this spans more than 3 files and more than 5 commits.



README rendered on GitHub.

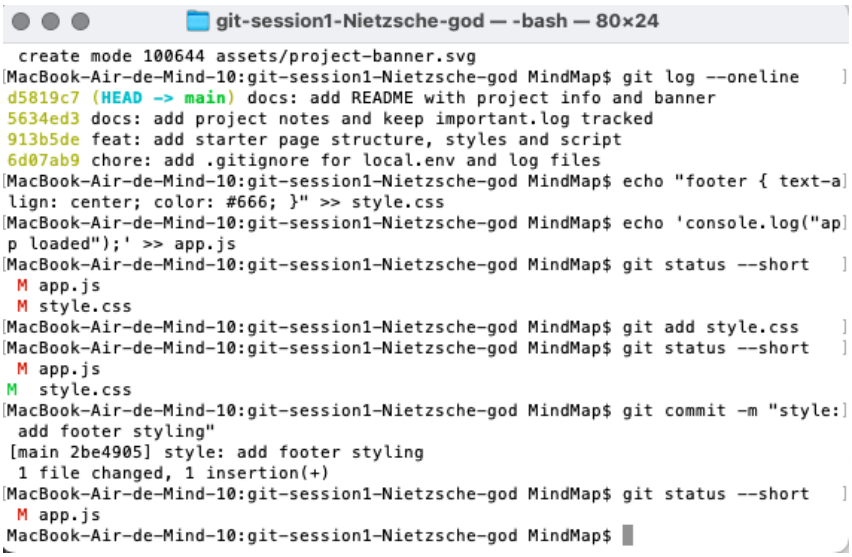
```
Writing objects: 100% (3/3), 336 bytes | 336.00 KiB/s, done.
Total 3 (delta 2), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (2/2), completed with 2 local objects.
To https://github.com/Nietzsche-god/git-session1-Nietzsche-god.git
   2511e7f..7f4f3c6  main -> main
MacBook-Air-de-Mind-10:git-session1-Nietzsche-god MindMap$ git status
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean
MacBook-Air-de-Mind-10:git-session1-Nietzsche-god MindMap$ git log --oneline --graph --decorate --all
* 7f4f3c6 (HEAD -> main, origin/main) feat: log message when app loads
* 2511e7f Update README.md
*   7b3ffcd Merge feature/about into main
|\
| * 1013a41 (feature/about) feat: add About section
|/
* 2be4905 style: add footer styling
* d5819c7 docs: add README with project info and banner
* 5634ed3 docs: add project notes and keep important.log tracked
* 913b5de feat: add starter page structure, styles and script
* 6d07ab9 chore: add .gitignore for local.env and log files
MacBook-Air-de-Mind-10:git-session1-Nietzsche-god MindMap$
```

Compact final history: `git log --oneline --graph --decorate --all`.

## 3. Selective Staging

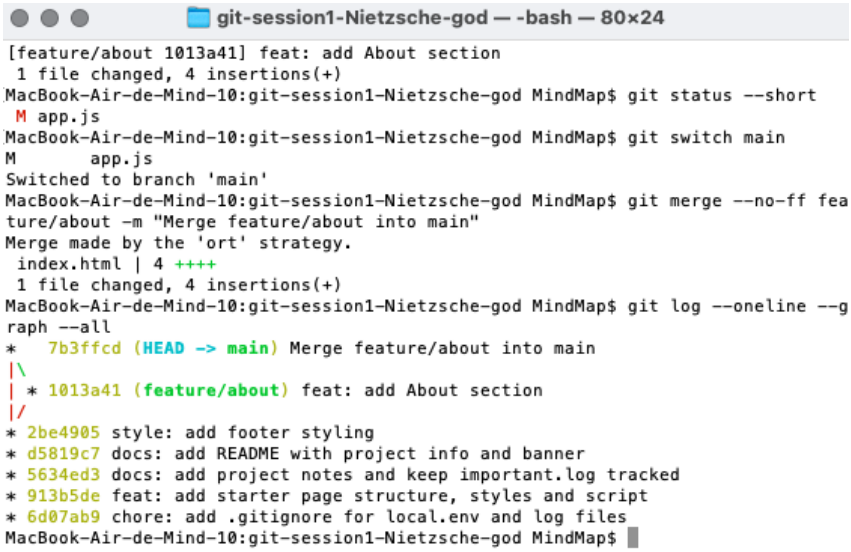
I appended a line to `style.css` and one to `app.js`, so both showed as modified. I ran `git add style.css` only, so `git status --short` showed `style.css` as staged and `app.js` as modified but not staged. I committed with the message "style: add footer styling", and `app.js` was still left modified. I kept it uncommitted until the diff step later.



Staged style.css was committed; app.js remained modified afterwards.

## 4. Branch & Merge

I created `feature/about` with `git switch -c`, added an About section to `index.html`, and committed it there. I then switched back to `main` and ran `git merge --no-ff feature/about`. The `--no-ff` option creates a merge commit so the branch stays visible in the history graph. My unstaged `app.js` change carried across both switches without conflicts because the branch only touched `index.html`.



Branch and merge history.

## 5. Remote Changes

**Commit vs push:** `git commit` records a snapshot in my *local* repository only. `git push` uploads those local commits to the remote (GitHub) so others can see them. **Fetch vs pull:** `git fetch` downloads new commits from the remote and updates `origin/main`, but does not change my branch or working files. `git pull` is fetch followed by merging the remote changes into my current branch. After I edited README.md on GitHub, `git fetch` followed by `git status` showed my branch was behind by 1 commit and my README was still unchanged. `git pull` then fast-forwarded it. Finally, `git diff`

showed the unstaged app.js change, and `git diff --staged` showed the same change after `git add app.js`. I then committed and pushed it.

```
remote: Resolving deltas: 100% (7/7), done.
To https://github.com/Nietzsche-god/git-session1-Nietzsche-god.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
MacBook-Air-de-Mind-10:git-session1-Nietzsche-god MindMap$ git fetch
remote: Enumerating objects: 5, done.
remote: Counting objects: 100% (5/5), done.
remote: Compressing objects: 100% (3/3), done.
remote: Total 3 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
Unpacking objects: 100% (3/3), 971 bytes | 161.00 KiB/s, done.
From https://github.com/Nietzsche-god/git-session1-Nietzsche-god
   7b3ffcd..2511e7f  main       -> origin/main
MacBook-Air-de-Mind-10:git-session1-Nietzsche-god MindMap$ git status
On branch main
Your branch is behind 'origin/main' by 1 commit, and can be fast-forwarded.
  (use "git pull" to update your local branch)

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   app.js

no changes added to commit (use "git add" and/or "git commit -a")
MacBook-Air-de-Mind-10:git-session1-Nietzsche-god MindMap$
```

After fetch: local main is behind origin/main by 1 commit (local files unchanged).

```
no changes added to commit (use "git add" and/or "git commit -a")
MacBook-Air-de-Mind-10:git-session1-Nietzsche-god MindMap$ git diff
diff --git a/app.js b/app.js
index 603d28a..9190aa5 100644
--- a/app.js
+++ b/app.js
@@ -3,3 +3,4 @@ const statusMessage = document.getElementById("status-message");
 if (statusMessage) {
   statusMessage.textContent = "Starter project is ready.";
 }
+console.log("app loaded");
MacBook-Air-de-Mind-10:git-session1-Nietzsche-god MindMap$ git add app.js
MacBook-Air-de-Mind-10:git-session1-Nietzsche-god MindMap$ git diff --staged
diff --git a/app.js b/app.js
index 603d28a..9190aa5 100644
--- a/app.js
+++ b/app.js
@@ -3,3 +3,4 @@ const statusMessage = document.getElementById("status-message");
 if (statusMessage) {
   statusMessage.textContent = "Starter project is ready.";
 }
+console.log("app loaded");
MacBook-Air-de-Mind-10:git-session1-Nietzsche-god MindMap$
```

Unstaged diff, then staged diff of app.js.

## 6. Fair Use of AI

I used Claude (Anthropic's AI assistant) as a guide while doing this assignment. I uploaded the assignment PDF and starter files and asked it to walk me through the workflow step by step on a Mac. Later questions were practical: whether to use VS Code or the Terminal, what to enter for `git config user.name`, how to restart the terminal after installing the `code` command, and what my command output meant. I used its suggested commands, but replaced the placeholders with my own name and GitHub username, adapted the README and About text it suggested, and edited the README on GitHub myself. I ran every command myself and verified the results with `git status`, `git log --graph`, `git check-ignore -v` and by checking the repository on GitHub. For example, the AI first said `check-ignore` would not print a line for `important.log`; my actual output showed the `!important.log` negation rule, and it corrected itself, which helped me understand that the rule is listed because the file is explicitly un-ignored. The AI also drafted the text of this report from my commands and screenshots; I reviewed it against my own repository before submitting.

## 7. Sources & Reflection

Sources: course Session 1 material and assignment brief; Git documentation (git-scm.com/doc); GitHub Docs; Claude (Anthropic) for guided step-by-step help.

The hardest concept was the staging area: a file can be modified in the working directory, staged, and committed as three separate states, and fetch versus pull also fooled me at first because fetch changes `origin/main` but not my files. If Git behaves unexpectedly, I would first run `git status`, then `git log --oneline --graph --all` and `git diff` to see which branch I am on, what is staged, and how local and remote differ.